GOOGLE CLOUD AI BUILDER CUP 2026 · HACK2SKILL · JAPAC

Rakshak judge pitch and Q&A

The 3-minute and 90-second spoken pitches, the climax rule, the 60-second judge self-test and the Q&A for the AI Builder Cup 2026 judging

Prepared for: Lohith Devaramane
Date: 17 October 2026
Basis: README.md, docs/ARCHITECTURE.md, docs/DEMO_STRATEGY.md, docs/PRIVACY_TIERS.md, docs/SECURITY_PRIVACY.md and the market existence check

Contents

1. The 3-minute spoken pitch (word for word)
2. The 90-second version
3. The rule: the payment screen is the climax
4. The 60-second judge self-test
5. Q&A

> **One line:** Pixel can hear the call, the telecom FRI knows the payee, the bank sees the payment; nothing joins them at the moment the money moves. Rakshak does, on the bank's side of the stack where the liability sits.

1. The 3-minute spoken pitch (word for word)

Good afternoon. I am [name] from Team Rakshak, and this is a story about one tap.

In Singapore last year, 81.8 percent of scam cases were self-effected transfers. The victim made the payment themselves. In India, cyber fraud cost at least 22,495 crore rupees in 2025, and about a quarter of it was recovered. Digital-arrest scams alone, where a caller pretends to be the CBI and keeps the victim on the line, ran to 1,935 crore across 123,672 cases in 2024.

Every one of those losses passed through the same moment: the victim, still on the phone with the scammer, tapped Pay. And every defence we have today looks away at that moment. Pixel can hear the call, but it only warns and it knows nothing about the payee. The bank's fraud engine sees the payee, but it is blind to the call, and a coerced victim passes every step-up because they want to pay. The SMS alert arrives after the debit, when the mule account is already being emptied. Under MAS's Shared Responsibility Framework, the liability for that missed real-time control sits with the bank.

Rakshak is a guardian agent that lives inside the bank app. With the customer's opt-in it listens to the call, transcribing each speaker on its own Gemini Live session and scoring the coercion after every sentence: authority impersonation, threats, secrecy, urgency, a safe-account demand, an OTP request. At 85 percent it whispers to the customer, privately. The caller never hears it.

Then the customer taps Pay. In under a hundred milliseconds the bank's own policy fuses the call risk with the payee's mule risk, computed from the ledger: twelve payers in 48 hours, 96 percent cashed out in a week, four complaints on the 1930 helpline. The transfer is paused for thirty minutes. The screen changes first, and then Rakshak speaks, in Hindi, over the scammer's voice: "Sunita ji, this payment is paused. This account received money from twelve people in two days. Priya has been told." Her daughter's phone chimes at the same moment.

Fifteen seconds later an ADK agent lands with the evidence. The investigator looked up the payee and checked the caller's claim against official advisories: the CBI does not do digital arrests, with the source. The decision agent confirmed the hold and wrote the reasons in Sunita's language. The agent can make the bank stricter, never looser; the policy is the floor. And Sunita keeps control: she can type the amount and pay anyway, or ask Rakshak why, by voice, and get an answer.

On fifty labelled calls in six languages, including ten written specifically to evade keyword detection, precision and recall are one point zero, and the payment decision is right on 38 of 39 attempts. The one miss is a warn, on purpose, on a small payment to a clean account.

Everything runs on Google Cloud: five Gemini models, ADK 2.11, Cloud Run in Singapore, Secret Manager, with Firestore, BigQuery and Pub/Sub on the scale path. It costs cents per guarded call. It is live right now at the URL on the screen, and you can run the full scam in sixty seconds on your own phone.

Scam detection is a crowded field. Stopping the payment, with the evidence, in the customer's language, with the customer still in charge: that is the gap, and that is Rakshak. Thank you.

2. The 90-second version

Eighty-two percent of Singapore's scam cases last year were transfers the victim made themselves, while the scammer was still on the line. Pixel can hear that call but only warns. The bank sees the payee but not the call. The SMS comes after the money has gone.

Rakshak is a guardian inside the bank app. With the customer's opt-in it transcribes the call speaker by speaker with Gemini Live and scores the coercion after every sentence. When the customer taps Pay, it fuses that call risk with the payee's mule risk from the ledger and, in under a hundred milliseconds, pauses the transfer for thirty minutes, speaks to the customer in Hindi over the caller, shows the payee's history and tells the daughter. An ADK agent then confirms with evidence from official advisories; it can only make the decision stricter. The customer keeps a typed-amount override and a voice channel to ask why.

Fifty labelled calls in six languages, ten of them adversarial: precision and recall one point zero. Five Gemini models, ADK 2.11, Cloud Run. Live now; try it in sixty seconds.

3. The rule: the payment screen is the climax

A risk meter during a call is prior art. Pixel ships it in Hindi; five Indian hackathon repos built it this year. If the judge's lasting image of Rakshak is a gauge climbing, Rakshak is filed under "scam detection, seen it".

So in every pitch, demo and video: ten seconds on the meter, then get to Pay. The climax is the hold screen: the pause, the Hindi voice over the caller, the payee card with the ledger numbers and lineage, "Priya has been notified", the agent confirming with the advisory source, and the three exits. Everything before it is setup. Nothing after it should be longer than the hold itself.

When a judge asks a question during the demo, answer with the hold screen in view. Do not navigate away from it to show the architecture; the architecture is on slide 6 and in the fraud desk trace.

4. The 60-second judge self-test

From the README, on the deployed service, no install:

1. Open `https://rakshak-223197895518.asia-southeast1.run.app/?session=judge&user=demo-user` on a phone or laptop. Finish the three onboarding taps.
2. Tap **Test an incoming call**, choose **Parcel from Taiwan (CBI digital arrest, Hindi)**, tap **Auto-play script**. Watch the transcript, the risk gauge and the Hindi reasons.
3. When the toast says the caller is demanding payment, tap **Open UPI pay** (the VPA `cbi.verification@rakshakdemo` and 2,50,000 rupees are pre-filled), then **Pay**.
4. See the hold screen within a second, hear Rakshak in Hindi, and open `https://rakshak-223197895518.asia-southeast1.run.app/desk` for the case with the agent trace and the 1930 referral.

What to say while it runs: "Every payee handle ends in @rakshakdemo, which does not exist on UPI; nothing moves money. The daughter's phone is at /family if you want to see the alert arrive."

If the judge has a second phone: phone B opens `/call?session=judge2`, phone A opens `/?session=judge2&user=demo-user`; B taps Call, A answers, they talk, A pays. That is the live path with real speech.

5. Q&A

How do you get the call audio on a real phone? A bank app cannot listen to a cellular call. Correct, and that is why Rakshak is not a phone app. Android removed call-audio access for third-party apps in Android 10 and iOS never had it; only the OS vendor's own dialer can touch that stream, which is why Google's scam detection runs inside the Pixel phone app. Rakshak runs on the bank's side, where call audio is lawfully available through three channels: the bank's own in-app calls (banks already route "verified calls" through their apps; this is the two-phone flow you just saw), a telco audio fork with consent (FICO Scam Signal with Jersey Telecom and Barclays already uses telco call-state for exactly this purpose; our Twilio bridge is the same shape), and an OS-vendor feed, which is the integration we propose: Google is the one company that can wire Pixel's on-device detection to a hold inside GPay, and Rakshak is the bank-side half of that pairing.

Isn't this what Pixel already does? Pixel hears the call and shows a warning. It knows nothing about who is being paid, it cannot pause anything, nothing reaches the bank, it is off by default, and Pixel is under one percent of phones in India. Rakshak starts where Pixel stops: it joins the call semantics with the payee's mule risk at the moment of authorisation, pauses the transfer, speaks in the customer's language, alerts the family and files the case. The two are complementary; the Pixel signal would be one more input to our fusion.

Where does the mule-risk data come from, and is it real? The features are the payment provider's own ledger: how many distinct payers in 48 hours, what share of the week's inflow was moved out, dormancy before a burst, account age. In this prototype they are computed live from a synthetic UPI ledger stream, 12,500 transactions seeded over seven days and about 150 new ones a minute, using the same formula as the production SQL in `sql/payee_risk.sql`. The registry fields a bank gets from outside, the DoT Financial Risk Indicator, 1930 complaint counts and KYC tier, are static demo values. Every payee card carries a lineage record saying exactly that. No bank data was used; the numbers are synthetic. With a bank, the same SQL runs on their ledger in BigQuery.

Why not just block the payment? Because a guardian that blocks legitimate payments is switched off within a week, and because the customer is an adult. A hold is a 30-minute pause with the evidence on screen, a voice channel to ask why, and a one-tap override that requires typing the amount. The override is recorded, the fraud desk still reviews the payee, and the hold expires on its own. Singapore's banks already use a 24-hour cooling-off for large balance movements; Rakshak makes the trigger the content of the call rather than the size of the balance, and keeps the customer in charge.

What about false positives and the override? Three layers. First, the rubric is told that urgency alone is not a scam, and the evaluation set has ten hard negatives (a real police station, a hospital deposit, a harsh landlord, a genuine fraud-desk call that says "OTP" and "blocked") that all score 0.10 or below. Second, the fusion needs both terms: a suspicious call to a clean payee for a small amount gets a warn, not a hold; that is the one "miss" in our evaluation and it is by design. Third, the override: typed amount, recorded reason, trusted contact already informed, case still reviewed. The field metric we most want and cannot have without users is the override rate.

Privacy, consent, DPDP, MAS? Guardian mode is opt-in with a visible "Rakshak ON" indicator; raw audio is never written to disk; the caller never hears Rakshak; the customer keeps the override; holds expire. The product is designed in three tiers so the customer chooses: Tier 0 uses no audio at all (call in progress plus payee risk), Tier 1 keeps a rolling transcript on the device and sends scores only when Pay is attempted, Tier 2, this prototype, analyses audio in the cloud where a lawful channel provides it. Under India's DPDP Act the bank is the data fiduciary, consent must be specific and withdrawable, and we keep only a transcript excerpt on the case for audit and the 1930 referral. Under MAS's Shared Responsibility Framework real-time surveillance and timely notification are the duties this implements; PDPA consent and retention apply as in India. Legal basis is a counsel question and we claim nothing more.

What if the scammer says "hang up and pay later"? Scoring runs from the first ring, not from the pay screen. When a call scores above the hold threshold, Rakshak records it on the customer's profile, and any payment within the next 60 minutes, even from a fresh app launch, is judged with that call's context, decayed with time.

A known low-risk payee in that window gets at most a warning; a new or risky payee gets the hold. The case shows the carry-over with the originating call and the minutes elapsed.

How does it scale? The only in-memory state is the live call; everything decided is written to the store. Cloud Run with WebSockets and session affinity handles the call; Firestore holds cases, profiles and alerts across instances; Pub/Sub fans alerts out to SMS, push or a dialler; the ledger streams to BigQuery and `sql/payee_risk.sql` runs as a scheduled query, materialised for sub-10 ms lookups; the ADK pipeline is self-contained and moves to Vertex AI Agent Runtime with profiles in Memory Bank. The work per call is I/O-bound on Gemini: two Live sessions and a flash call every three seconds.

What is synthetic and what is real? Real: the five Gemini models, the ADK pipeline, the live transcription of real speech over the two-phone relay, the Cloud Run deployment, the latency numbers measured on the public URL, the official-advisory table with its sources. Synthetic: the 50 evaluation calls, the 12 demo scripts, the 64 payees, the ledger stream, the two demo customers. Not implemented: Tier 1 on-device, Agent Runtime, Memory Bank, a bidirectional telco stream. We say which is which on every slide.

What would you do with a bank partner? Three things in ninety days. Run `sql/payee_risk.sql` on their ledger with the DoT FRI, I4C and KYC feeds so the payee term is real. Build a human-labelled corpus of consented in-app calls, 500 calls with 100 confirmed scams and 100 hard negatives from their complaints desk, and calibrate the warn and hold thresholds per customer segment to their loss and friction targets. Pilot behind Firebase Auth and App Check with override and outcome tracking, so precision is measured on consequences.

Cost per call? Per guarded three-minute call: two Live transcription sessions, ten to twenty rubric calls on gemini-3.5-flash, one cached voice clip, one ADK run of two agents and two or three tool calls, one narrative call. At flash-tier list prices that is cents, against a transfer of two lakh rupees. The always-on Cloud Run instance is tens of dollars a month and is the dominant fixed cost for a demo; in production it is amortised across every call.

Why Gemini 3.5 for the rubric and 3.8 for the agents? We measured. The rubric runs after every sentence, so it must return in the gap between utterances. gemini-3.5-flash at minimal thinking returns the strict JSON rubric with clean Devanagari, Kannada and Tamil reasons in about 2.4 to 3 seconds. gemini-3.8-flash took about 8 seconds and rejects the fastest thinking level, which is too slow between utterances; gemini-3.5-flash-lite was faster but garbled Devanagari. The agents are off the critical path, because the deterministic policy has already acted, so they get the model with the best tool use, gemini-3.8-flash, with automatic fallback to 3.5-flash. TTS is 3.8-flash-tts for Hindi prosody, prewarmed per amount because synthesis takes about six seconds. The post-hold conversation is 3.8-live because it needs audio out with function calling.

Why a deterministic policy at all if you have an agent? Because the hold must happen before the customer's thumb lifts. The policy is pure arithmetic over the rubric, the payee row and the profile, runs in under 100 ms, is unit-tested, and is the floor: `final = max(agent, policy)`. The agent adds evidence, the advisory source and a message in the customer's language, and can upgrade; it cannot release. A failed or slow agent leaves the policy in force. That bounds a model error to "slightly more cautious".

What happens when the Live API or a model is down? Each piece degrades on its own: the transcriber reconnects with backoff and the typed-turn path still works; a failed rubric keeps the previous score; slow TTS never delays the hold because the screen changes first; a failed agent leaves the policy decision; BigQuery falls back to the local table and then to a thin-file default; Pub/Sub or SMS failure still records the in-app alert. The demo runs with zero Google Cloud data services configured and a keyword stub if there is no key at all.